

第4章 存储管理

4.6 虚拟存储器的基本概念

- 存储管理的两个基本问题：
 - 大作业如何在小主存上运行；
 - 如何在给定大小主存上运行更多程序；

4.6.1 虚拟存储器的理论基础

(1) 程序运行的基本特征:

局部性原理:在一段时间内一个程序的执行往往呈现出高度的局部性,表现在时间与空间两方面 [Denning P. 1968]:

□ 时间局部性:

一条指令被执行了,则在不久的将来它可能再被执行;
在一段时间内,访问的代码范围是有限的。

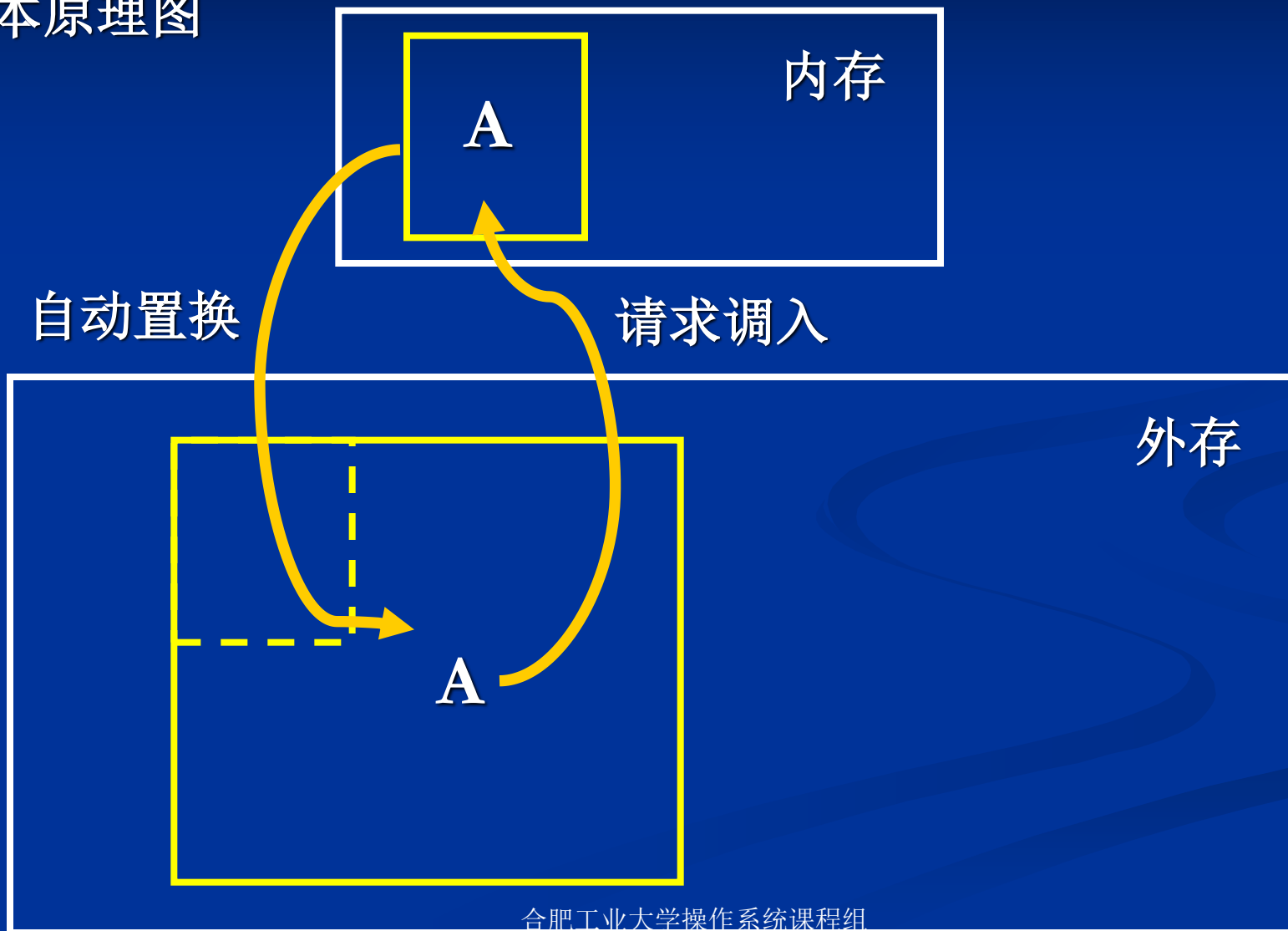
□ 空间局部性:

若某一存储单元被使用,在一定时间内,与该存储单元相邻的单元可能被使用

■ 启示: 是否可不将程序所有代码同时装入主存?

4.6.2 虚拟存储器概念

(1) 基本原理图



4.6.2 虚拟存储器概念

(2) 虚拟存储器的定义：

- 所谓虚拟存储器，是指具有请求调入功能和置换功能，能从逻辑上对内存容量加以扩充的一种存储器系统。
- 其逻辑容量由内存容量和外存容量之和所决定，其运行速度接近于内存速度，而每位的成本却又接近于外存。
- 虚拟存储技术是一种性能非常优越的存储器管理技术，故被广泛地应用于大、中、小型机器和微型机中。

(3) 虚拟存储器的特征：

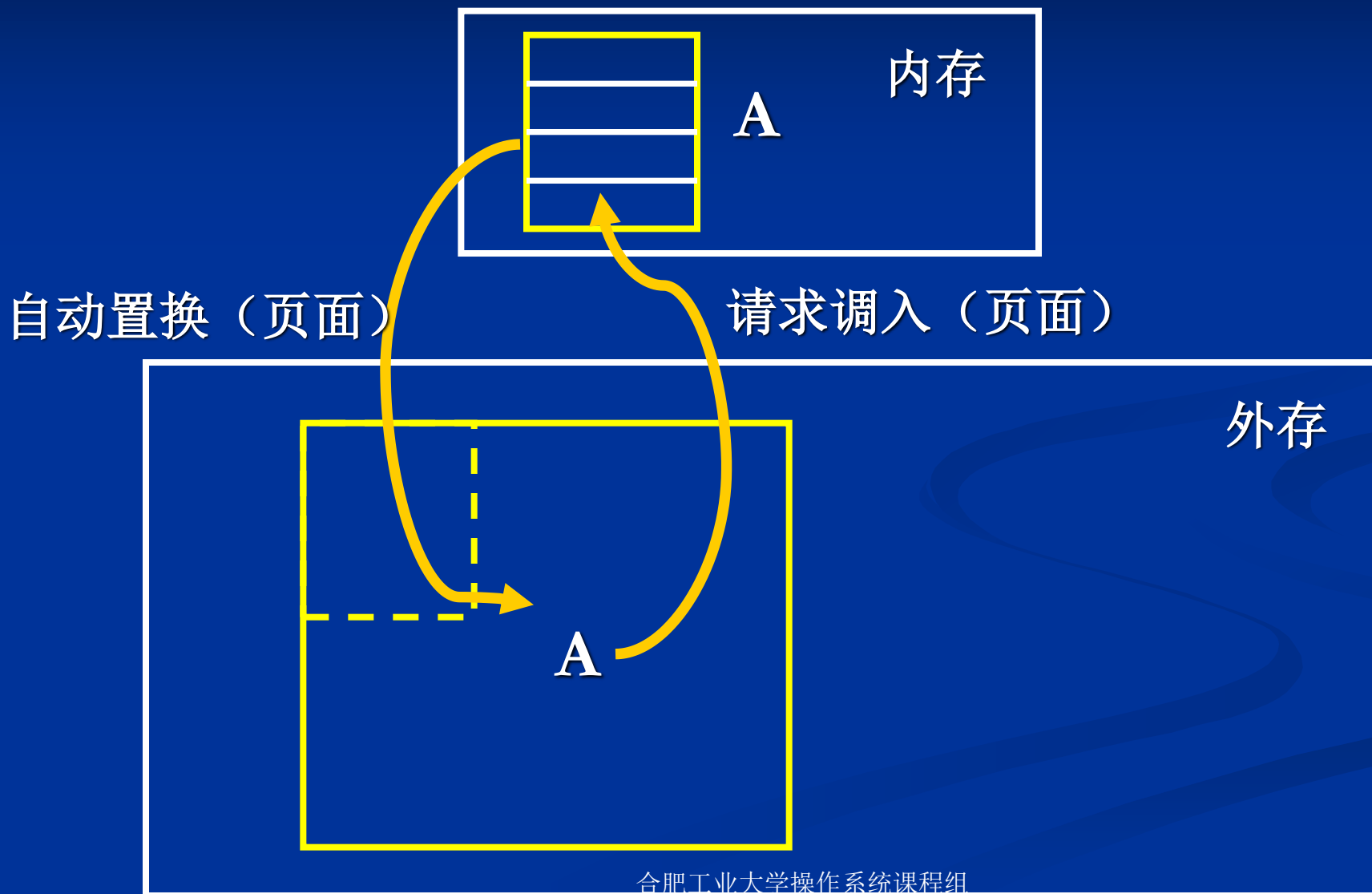
- 多次性。
- 对换性。
- 虚拟性。

4.6.2 虚拟存储器概念

(4) 虚拟存储器的实现:

- 分页请求系统（页式虚拟存储器，请求分页系统）：
在基本分页系统基础上，增加以页面为单位的请求调入和自动置换功能。
- 请求分段系统（段式虚拟存储器，分段请求系统）：
在基本分段系统基础上，增加以分段为单位的请求调入和自动置换功能。
- 段页式虚拟存储器：
在段页式系统基础上，增加以分页为单位的请求调入和自动置换功能。

4.7 请求分页存储管理方式

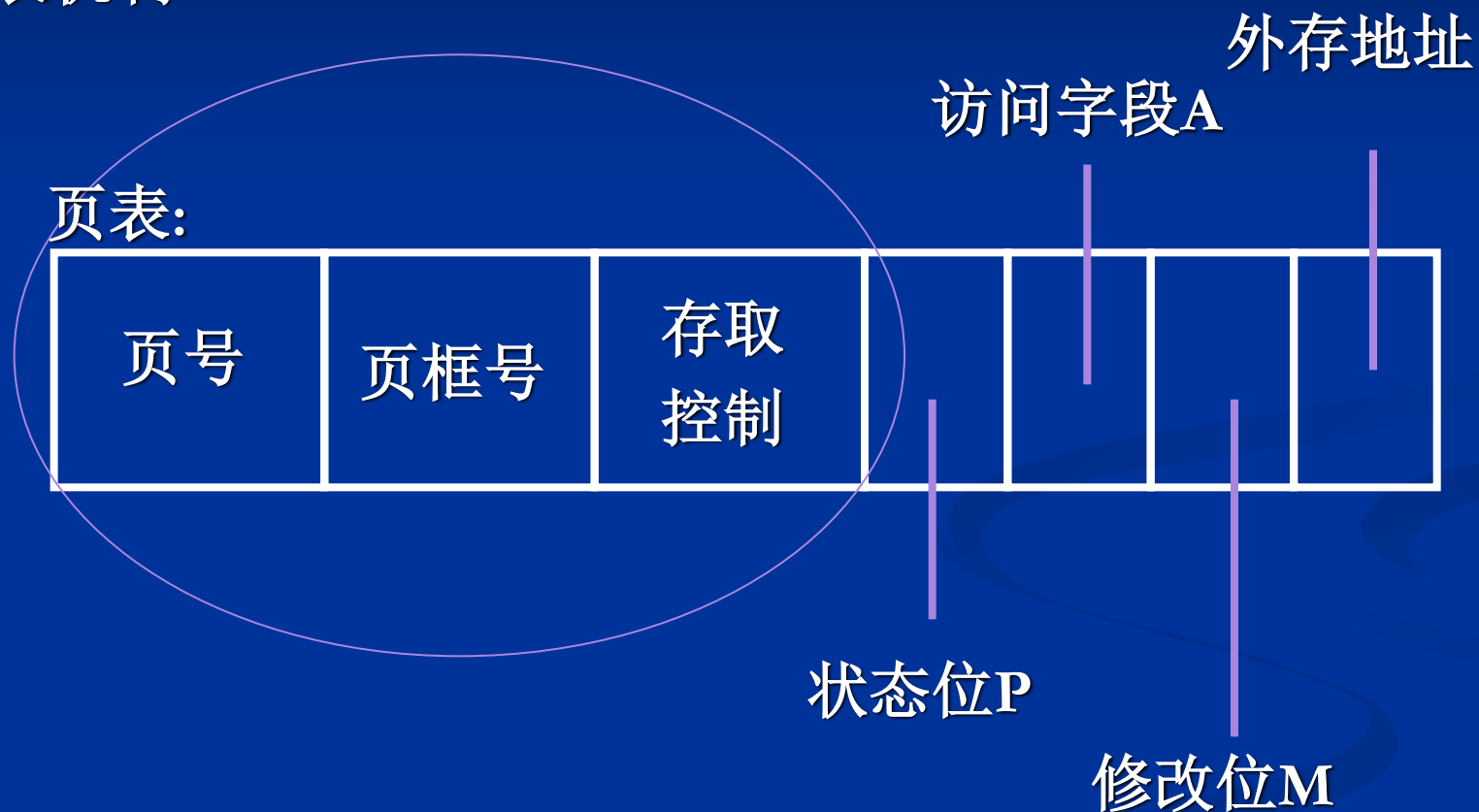


■ 要解决的问题:

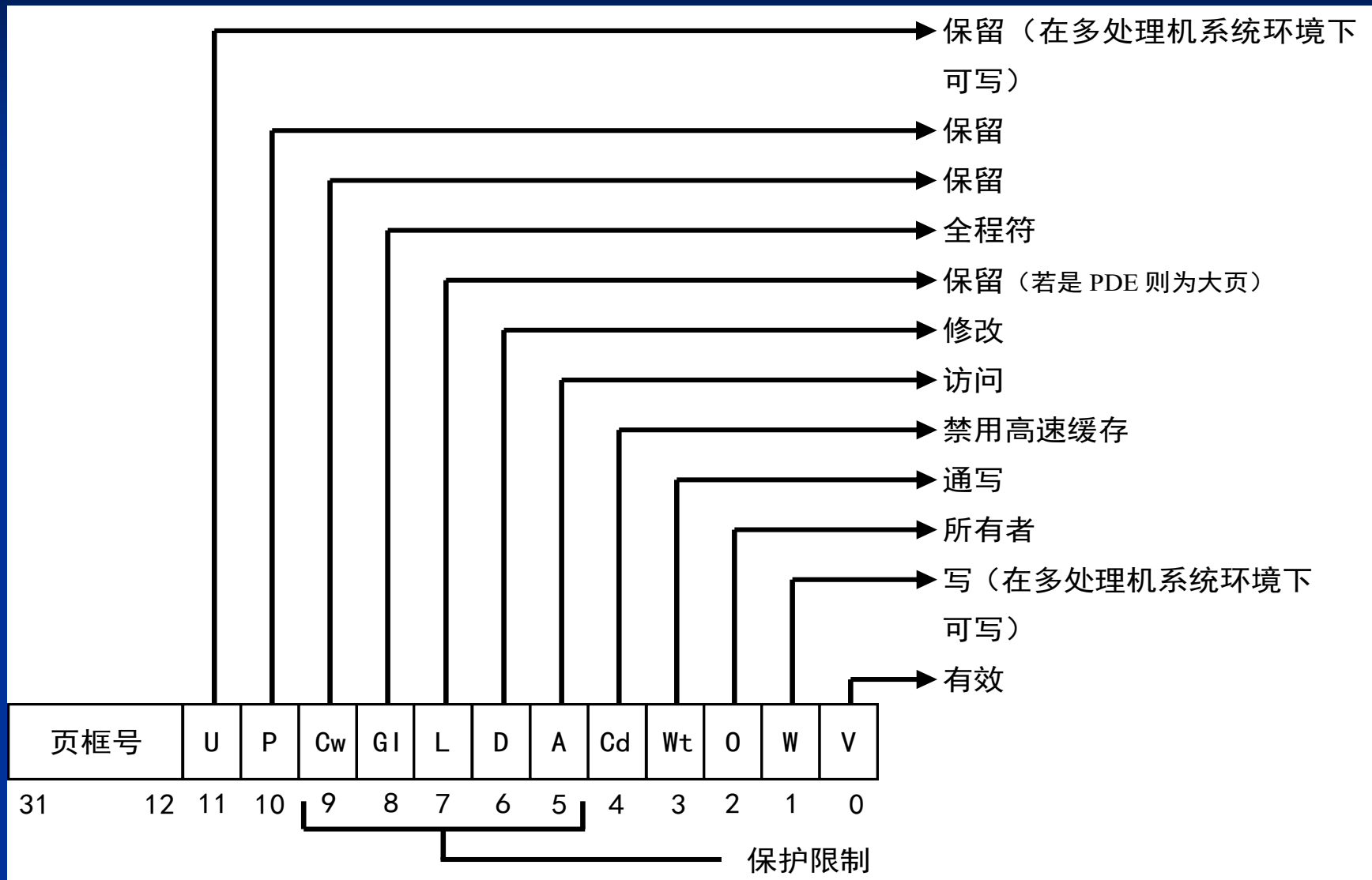
- 页面现在可能不在主存了，如何修改页表；
- 操作系统如何知道需要页面调入；
- 地址变换；
- 初始页框数量如何确定；
- 何时调入页面；
- 何处调入页面；
- 如何置换页面；

4.7.1 请求分页中的硬件支持

(1) 页表机制



Intel x86 CPU页表



4.7.1 请求分页中的硬件支持

(2) 缺页中断机制

■ 何时发出缺页中断

```
MOV AX, [1000];  CALL CS:[100];  JNZ CS:[2000]  
CS:IP
```

■ 缺页中断

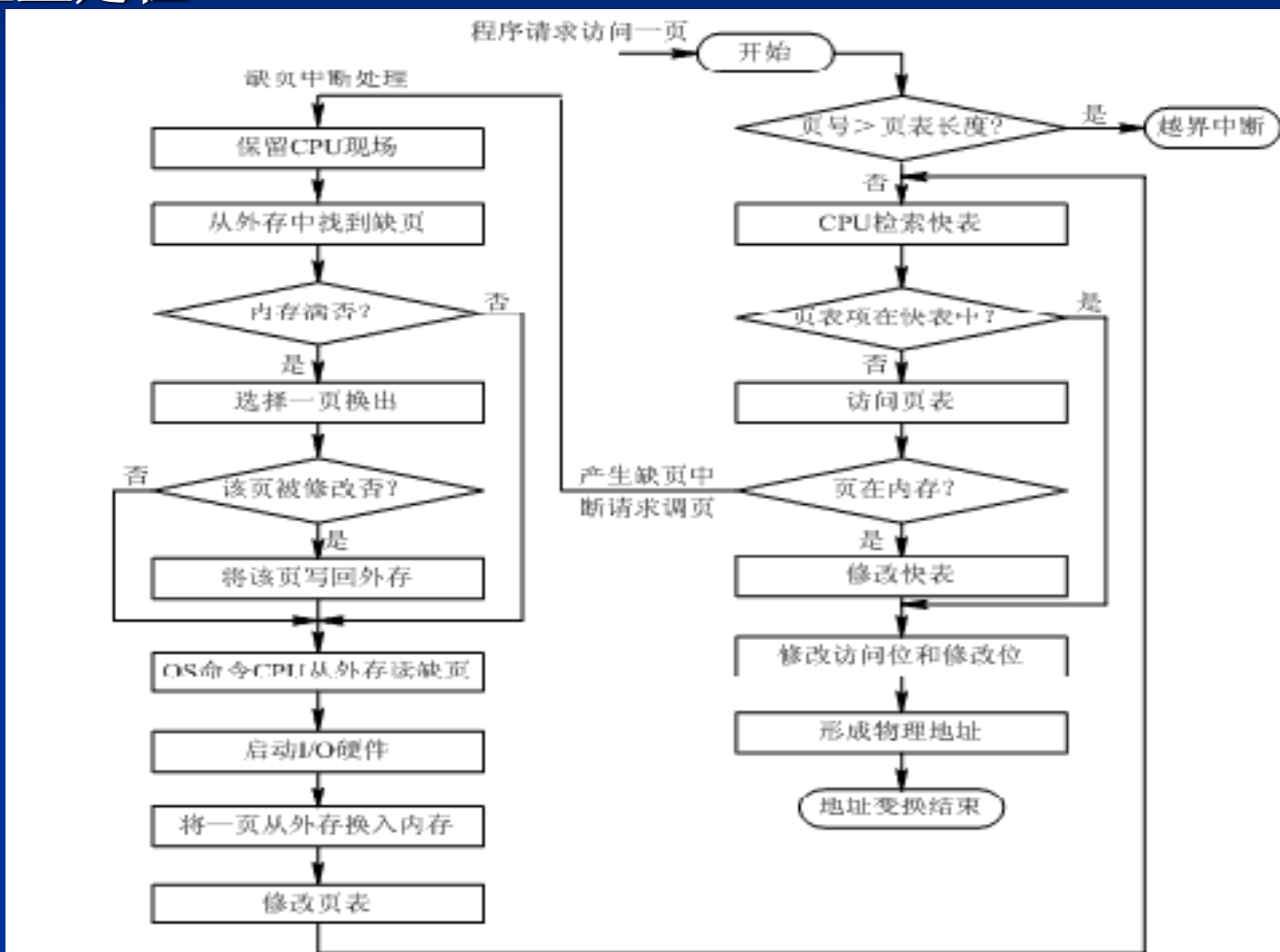
- 指令执行期间：发出中断并响应和处理中断，返回
- 一条指令可能发出多次缺页中断；

问题：Intel x86 CPU，为何没有下述指令？

```
MOV DS:[2000], CS:[1000]
```

4.7.1 请求分页中的硬件支持

(3) 地址重定位



4.7.2 内存分配策略和算法

(1) 最小物理块个数

- 保证程序正常运行需要的最小物理块个数:

由机器指令的结构决定

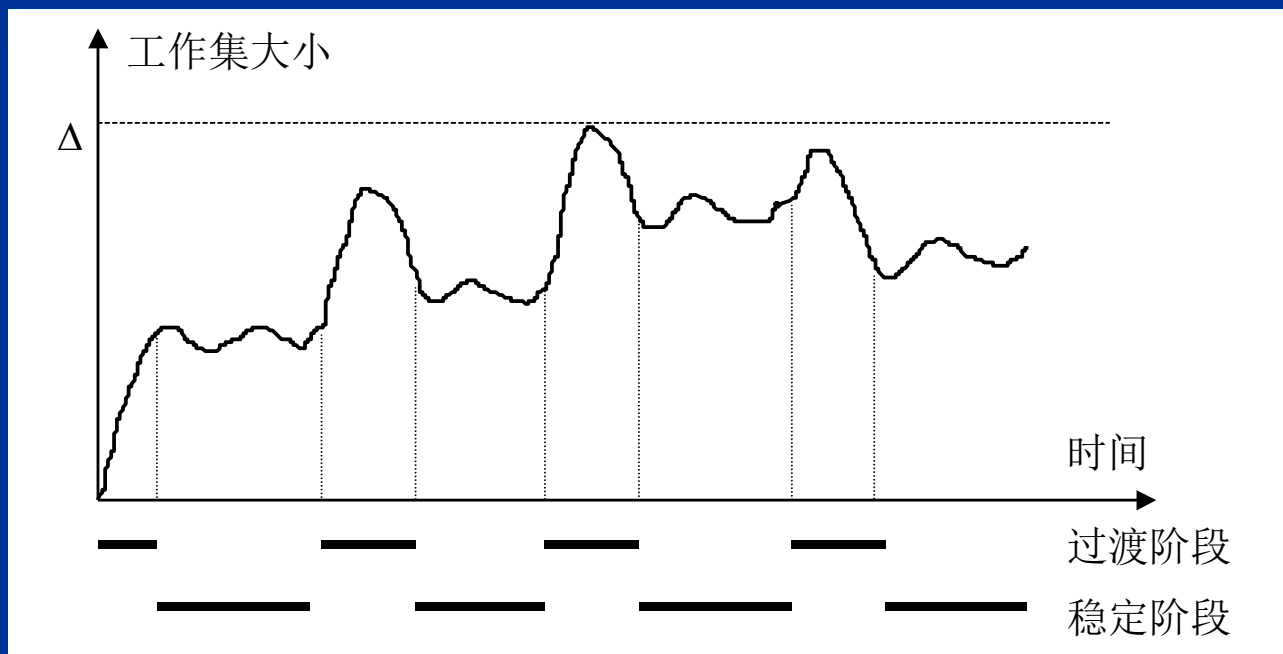
- 单字节指令，直接寻址：2块；
- 单字节指令，间接寻址：3块；
- 多字节指令，直接寻址：3块；
- 多字节指令，间接寻址：4块；

4.7.2 物理块分配策略和算法

(2) 工作集 [Denning P.]：进程执行时需要多少物理块？

■ 进程的工作集

驻留在物理内存中的虚拟页面的子集。



工作集大小的变化：进程开始执行后，随着访问新页面逐步建立较稳定的工作集。当内存访问的局部性区域的位置大致稳定时，工作集大小也大致稳定；局部性区域的位置改变时，工作集快速扩张和收缩过渡到下一个稳定值

4.7.2 内存分配策略和算法

(3) 物理块分配策略

■ 固定分配局部置换

- 为进程分配物理块;
- 在进程运行期间, 物理块个数不变;
- 新页面, 只能置换入该进程已经分配的物理块;

■ 特点

- 物理块个数难以指定;

4.7.2 内存分配策略和算法

(3) 物理块分配策略

■ 可变分配全局置换

- 为进程分配物理块;
- 在进程运行期间, 物理块个数随时变化;
- 新页面, 在全局范围内置换物理块;

■ 特点

- 进程之间的执行互相影响;
- 页面“抖动”

4.7.2 内存分配策略和算法

(3) 物理块分配策略

■ 可变分配局部置换

- 为进程分配物理块;
- 在进程运行期间, 物理块个数可变化;
- 新页面, 在该进程已经分配物理块内置换;
- 引入缺页率, 保证:

低限阈值 $\leq$ 缺页率 $\leq$ 高限阈值

■ 特点

- 避免物理块初始设置不合理的难题;

4.7.2 内存分配策略和算法

(4) 物理块分配算法

- 平均分配
- 按比例分配
- 加权分配

4.7.3 页面策略

(1) 何时调入

- 预先调页
 - 一次调入多个页面;
 - 减少后期I/O;
- 请求式调页
 - 需要时, 调入;

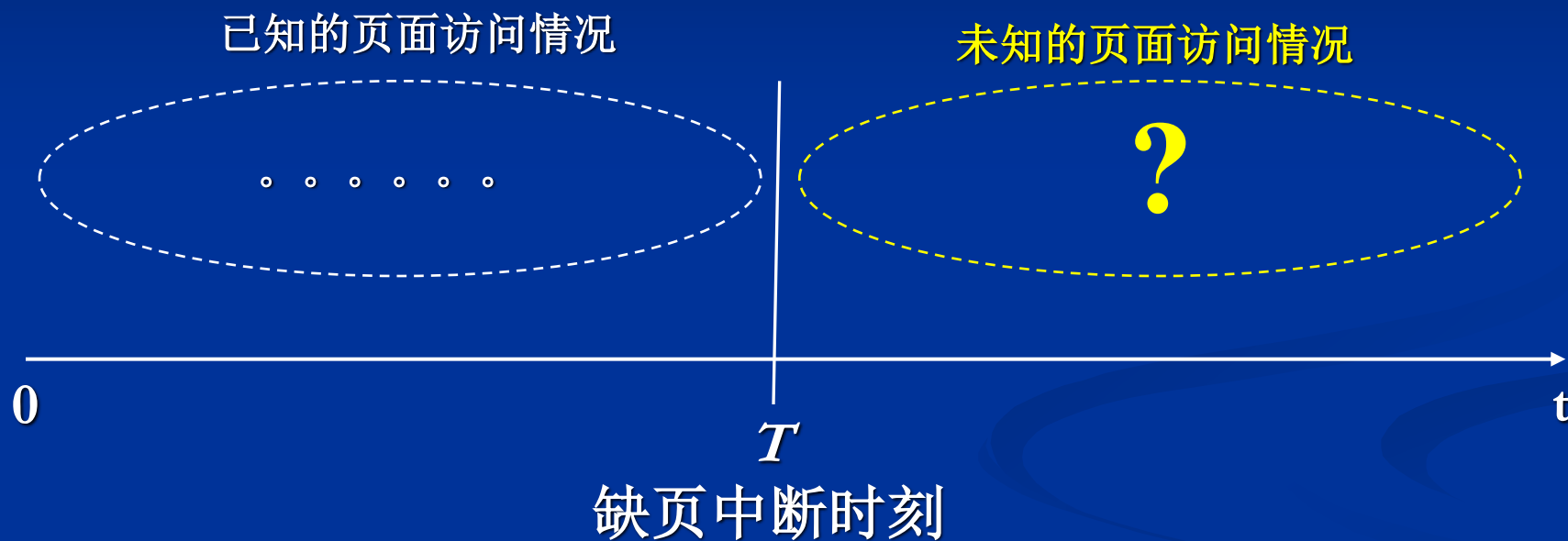
(2) 何处调入

- 外存对换区
- 文件区
- UNIX方式: 文件区(第一次); 对换区(兑换区);

4.7.4 页面置换算法

- 最佳置换算法(OPT算法)
- 先进先出算法(FIFO算法)
- 最久未使用页面置换算法(LRU算法)
- Clock置换算法

■ 页面置换/淘汰：Cache更新---预测未来的页面访问情况



4.7.4 页面置换算法

(1) 最佳置换算法

■ 基本思想

- 选择以后再也不用的页面；
- 没有的话，选择以后最长时间不用的页面；

■ 实现

- 无法实现，因为页面的访问顺序无法预知；

■ 特点

- 无法实现，仅具有理论意义；

4.7.4 页面置换算法

(2) 先进先出置换算法(FIFO)

■ 基本思想

- 基于：程序的顺序执行特点
- 选择到达内存最早的页面，予以淘汰；

■ 实现

- 页面在内存中按时间排序；

■ 特点

- 效果不佳（程序不是严格顺序执行）；

4.7.4 页面置换算法

(3) 最近最久未使用置换算法(LRU)

■ 基本思想

- 基于：程序运行的局部性原理；
- 选择最近以来最久未使用的页面，予以淘汰；

■ 实现

- 移位寄存器，栈；

■ 特点

- 调度性能教好；

4.7.4 页面置换算法

(3) 最近最久未使用置换算法(LRU)

■ 移位寄存器实现方案

- 每个物理块设置一个移位寄存器，初值为0；
- 访问一次页面，高位置1；
- 定时（100ms）将所有物理块的移位寄存器右移1位，高位补0；
- 选择移位寄存器数值最小的物理块，淘汰其中页面

4.7.4 页面置换算法

(3) 最近最久未使用置换算法(LRU)

■ 栈实现方案

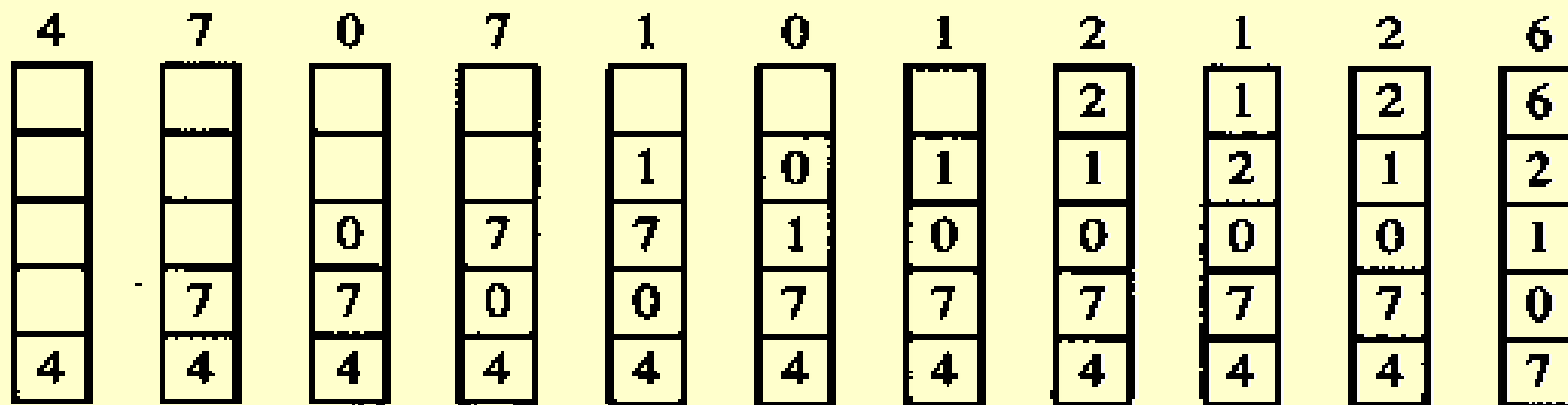


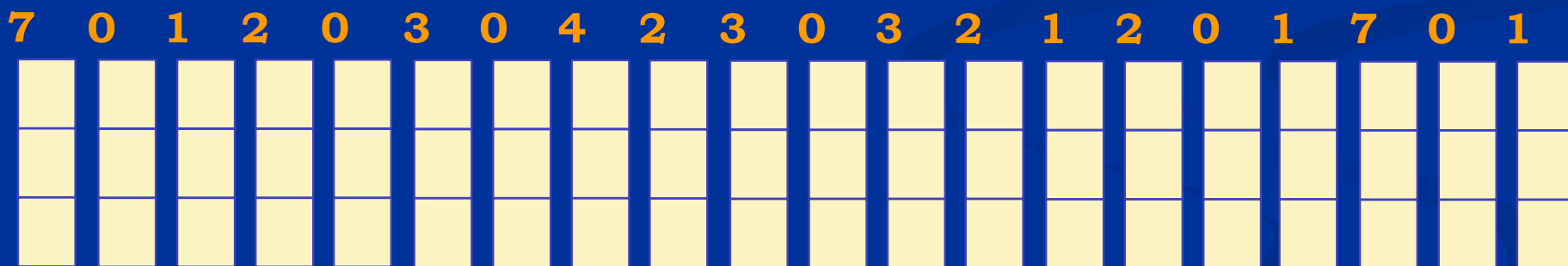
图 4 - 29 用栈保存当前使用页面时栈的变化情况

4.7.4 页面置换算法

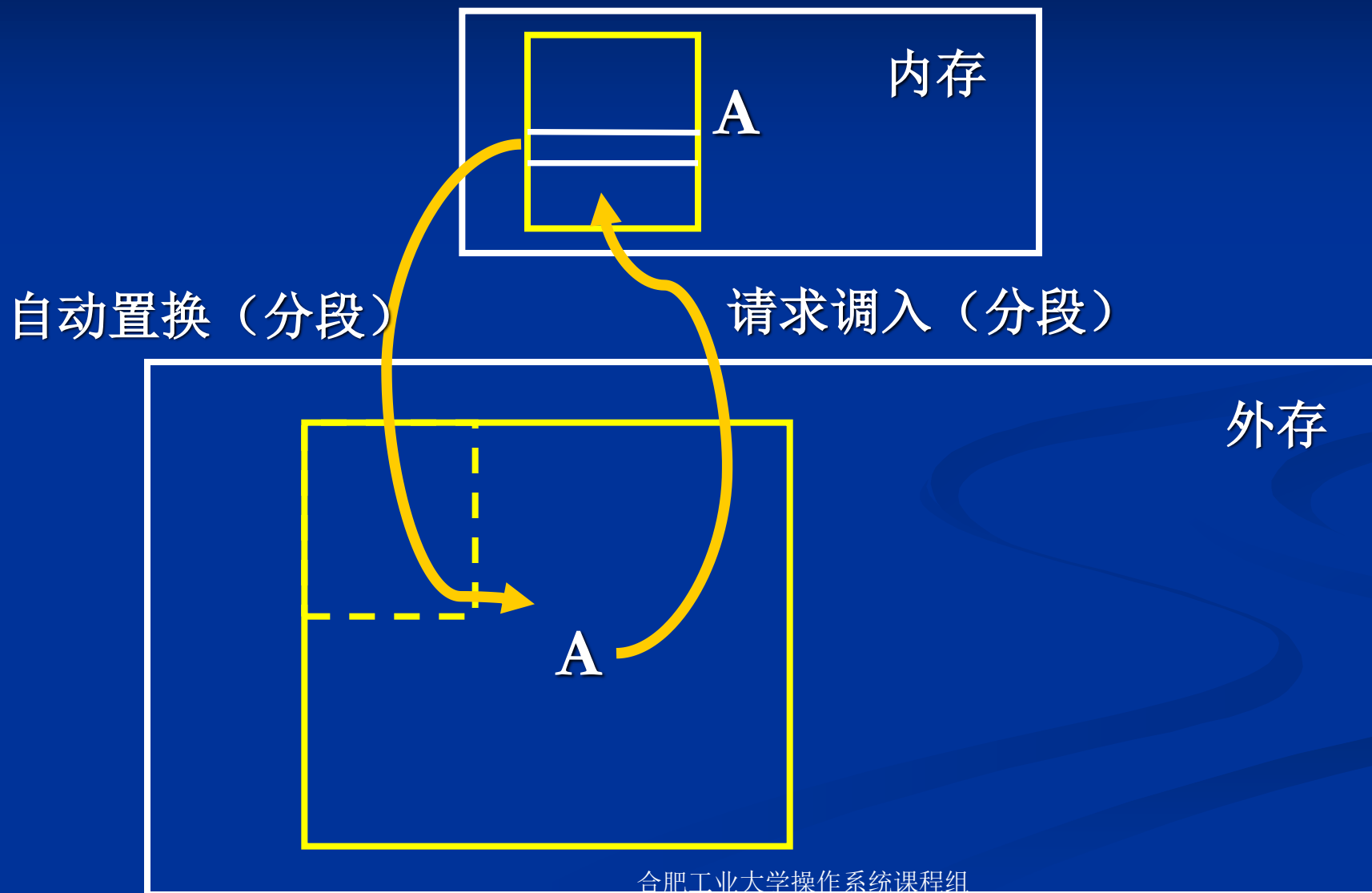
(4) 置换算法性能比较

■ 实例1:

某分页系统，进程分到3个存储块，其页面访问序列为：7-0-1-2-0-3-0-4-2-3-0-3-2-1-2-0-1-7-0-1，请分别计算三种页面调度算法OPT、FIFO、LRU下的缺页率（缺页率=缺页置换次数/页面访问总数*100%）。



4.8 请求分段存储管理方式



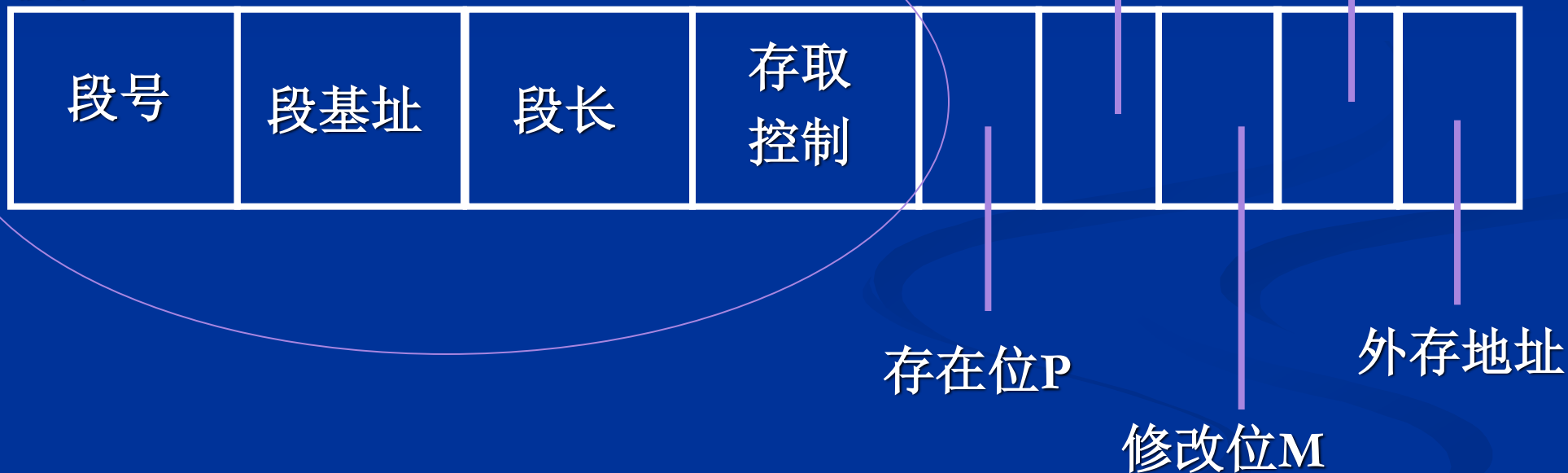
■ 要解决的问题:

- 分段现在可能不在主存了，那么如何修改段表；
- 操作系统如何知道需要分段调入；
- 地址变换；
- 如何实现分段共享？

4.8.1 请求分段中的硬件支持

(1) 段表机制

段表:



4.8.1 请求分段中的硬件支持

(2) 缺段中断机制

■ 何时发出缺段中断

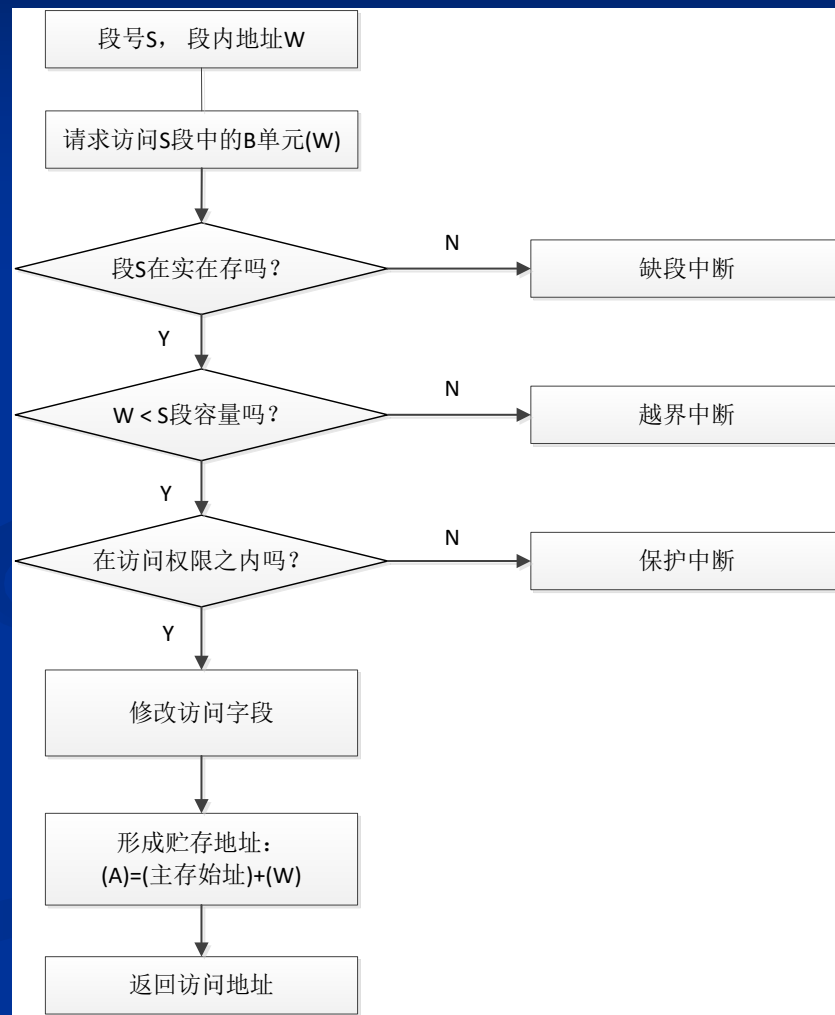
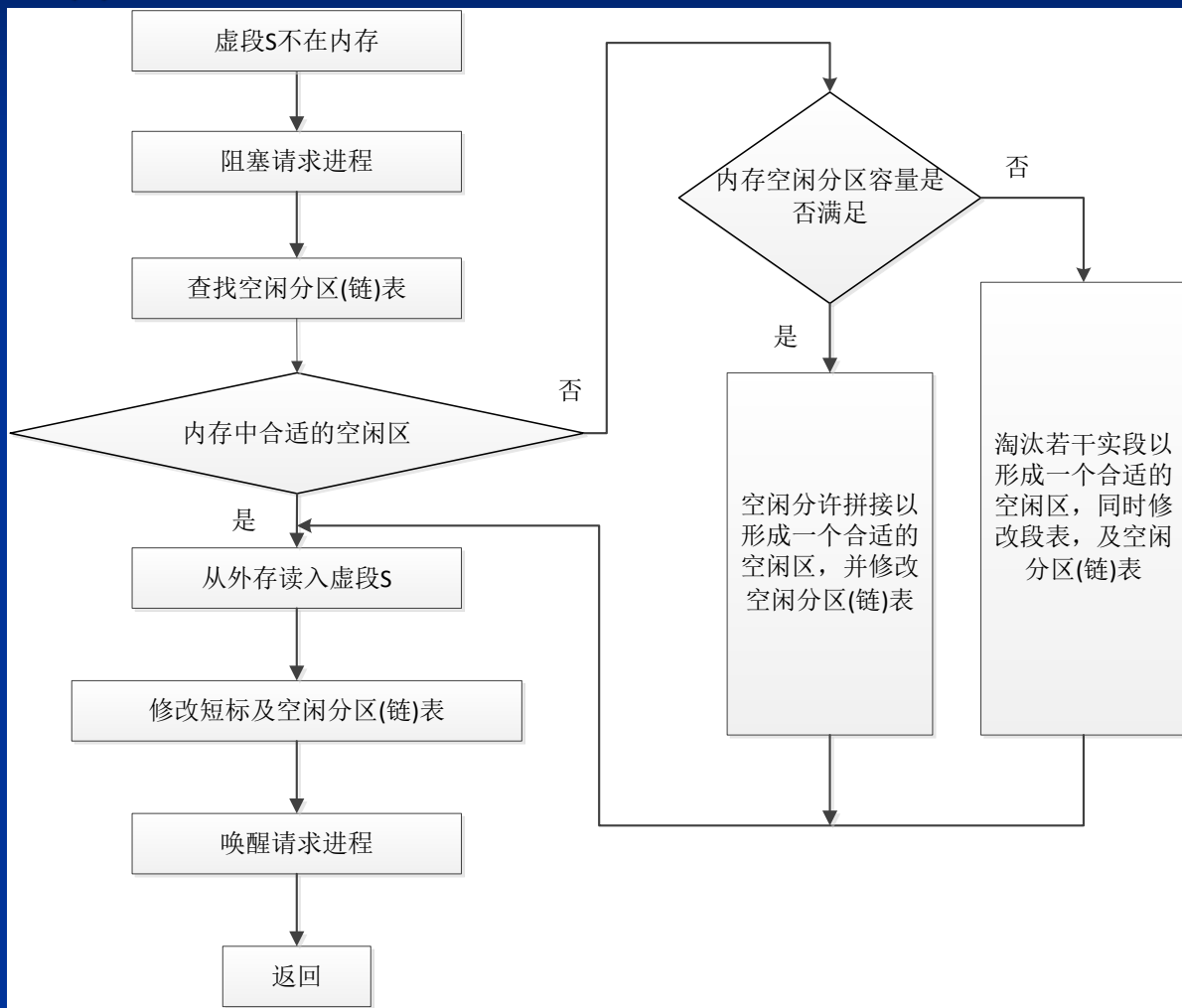
MOV AX, [1000]; CALL CS:[100]; JNZ CS:[2000];
CS:IP

■ 缺段中断

- 指令执行期间：发出中断并响应和处理中断，返回
- 一条指令可能发出多次缺段中断；

4.8.1 请求分段中的硬件支持

(3) 地址重定位



4.8.2 分段共享与保护

(1) 数据结构

- 共享段表（所有进程共享）

整个系统设置1张共享段表；

记录：分段信息，共享该分段进程的信息(存取控制，进程数量count，段号)。

- 进程段表（私有段表）

与前面所述段表相同；

注：被共享分段可由各进程以不同方式（存取控制，段号）共享。

4.8.2 分段共享与保护

(2) 算法：共享分段的分配与回收

■ 分配算法

- a) 检索共享分段，如果共享分段存在转c)；
- b) 创建共享段表的段表项，填写分段信息，分配相应内存并调入分段；
- c) 记录本进程信息到共享段表的段表项, $count+1$ ；
- d) 复制共享段表的段表项相应信息（段基址、段长、存取控制）到进程段表；

■ 回收算法

- a) 撤消进程时，将所有该进程共享之分段的 $count-1$ ；
- b) 当某分段 $count = 0$ 时，回收该分段所占据内存；